MyCodeSchool

Spring Boot Fundamentals (Lecția 01)

Dependency Injection, ApplicationContext, stereotypes, `@Configuration` + `@Bean`, `@Qualifier`, `@Primary`, `@Value`, `@PostConstruct`, `@Scope`, `@Profile`.

Fără web, fără DB — doar containerul Spring și injecția de dependențe. Un singur proiect Maven; toate comenzile se rulează din rădăcina lui.

Cerințe

```
java -version    # 21 (folosește ./mvnw — nu-ți trebuie Maven instalat)
```

Structură

unde	ce e	teste
`teorie/`	de citit înainte: `TEORIE.pdf`, `GRESELI-COMUNE.pdf`, `EXERCITII-TEORIE.pdf`	—
`src/main/java/com/example/demo/`	exemple rezolvate: vezi Spring lucrând pas cu pas	✅ verzi
`src/main/java/com/example/fundamentals/`	27 de exerciții de făcut tu: `ex01-ex07` → `practica` → `provocari`	🔴 până le rezolvi

unde	ce e	teste
`src/main/java/com/example/notifications/`	challenge: sistem de notificări, schelet + teste de acceptanță	🔴 până îl rezolvi
`src/main/resources/application.yaml`	configurarea comună	—

Ordinea recomandată

```
# 1. citește teoria
open teorie/TEORIE.pdf

# 2. vezi conceptele rulând concret în consolă
./mvnw spring-boot:run

# 3. fă exercițiile, nivel cu nivel
./mvnw test -Dtest='com/example/fundamentals/ex*/**'
./mvnw test -Dtest='com/example/fundamentals/practica/**'
./mvnw test -Dtest='com/example/fundamentals/provocari/**'

# 4. atacă challenge-ul (cerințele în notifications/CERINTE.pdf)
./mvnw test -Dtest='com/example/notifications/**'
```

Un singur test: `./mvnw test -Dtest=Ex01GreetingServiceTest`. Tot proiectul: `./mvnw test`.

> `RED` e starea de pornire — faci testele `GREEN` rezolvând exercițiile. **Nu modifici testele** — ele sunt contractul.

Exemplele rezolvate — `demo/` (biblioteca de cărți)

`./mvnw spring-boot:run` pornește aplicația, afișează fiecare pas al construirii contextului, apoi se închide (nu e server web). Domeniul e o bibliotecă: `Book`, `BookRepository` (depozit în memorie), `BookService`, `BookReport`.

```
[constructor] InMemoryBookRepository#1 (depozit gol)
[@PostConstruct] InMemoryBookRepository#1.seed() → 3 cărți
[constructor] PlainBookFormatter
[constructor] BookReport → a primit: InMemoryBookRepository#1, PlainBookFormatter
[@Bean] ClockConfig.clock() called — published Clock(zone=Europe/Bucharest)
[constructor] BookService → a primit: InMemoryBookRepository#1, Clock,
              libraryName='Biblioteca MyCodeSchool'
[constructor] CsvBookFormatter
```

Observă ordinea: tot ce e deasupra liniei `SPRING FUNDAMENTALS — DEMO` s-a întâmplat **înainte** să ruleze vreo linie scrisă de noi: Spring a scanat pachetul, a chemat constructorii, a rulat `@PostConstruct`. Abia la final `DemoRunner.run(...)` folosește bean-urile.

Demo-ul are 4 secțiuni, în ordinea în care merită înțelese:

Secțiune	Ce dovedește	Unde în `demo/`
1) Component scan	lista bean-urilor găsite în `com.example.demo`, cu adnotarea fiecăruia; `PaperBookPrinter` **n-are adnotare** → nu e bean, iar dacă o ceri într-un constructor aplicația nu pornește	`DemoRunner`, `PaperBookPrinter`
2) `@Component` = un singur `new`	`InMemoryBookRepository.instancesCreated() = 1`; `DemoRunner`, `BookService` și `BookReport` au primit **aceeași** instanță `#1`: adaugi o carte prin service, o vezi în report	`InMemoryBookRepository`, `BookService`, `BookReport`
3) Fără Spring	același cod cu `new` de mână → depozitele `#2` și `#3`, cartea adăugată într-unul nu există în celălalt, și niciunul n-are cele 3 cărți din `seed()`	`DemoRunner`, secțiunea 3
4) Restul adnotărilor	`@Value` (numele bibliotecii din yaml), `@Bean` (`Clock`, clasă JDK), `@Primary` (`PlainBookFormatter` câștigă în fața `CsvBookFormatter`), `@PostConstruct` (`seed()`)	`BookService`, `ClockConfig`, `PlainBookFormatter`, `CsvBookFormatter`

Ideea centrală: `@Component`, `@Service` și `@Repository` înseamnă același lucru: „Spring, fă tu `new` la clasa asta, o singură dată, și dă instanța oricui o cere prin constructor". Numele diferă doar ca să citești rolul clasei. Adnotarea **nu schimbă clasa** (secțiunea 3 cheamă aceiași constructori de mână), ci doar îi spune containerului ce să construiască. Fixul clasic de mână pentru „o singură instanță" e Singleton-ul cu `getInstance()`; cu `@Component` nu-l mai scrii.

Testele din `src/test/java/com/example/demo/` (6, verzi) verifică fix aceste afirmații: `ComponentScanTest` (clasa fără adnotare nu e bean), `RepositorySingletonTest` (service și report împart depozitul), `SeedPostConstructTest` (depozitul lui Spring are 3 cărți, cel făcut cu `new` are 0), `BookServiceTest`, `FormatterPrimaryTest`, `ClockBeanTest`.

De încercat: șterge `@Repository` de pe `InMemoryBookRepository` și pornește (`Parameter 0 of constructor in ...BookService required a bean of type 'BookRepository' that could not be found`) · pune `@Component` pe `PaperBookPrinter` și cere-o în constructorul lui `DemoRunner` · comentează `@Primary` din `PlainBookFormatter` (`NoUniqueBeanDefinitionException`) · comentează `@PostConstruct` din `InMemoryBookRepository` (depozitul `#1` pornește gol) · schimbă `demo.library.name` în `application.yaml`.

Nivel 1 — `fundamentals/ex01–ex07` (ghidate)

Fiecare clasă are un `// TODO` care-ți spune **exact** ce adnotare să adaugi.

#	Temă	Test
Ex01	`@Component`	`Ex01GreetingServiceTest`
Ex02	`@Service` / `@Repository`	`Ex02ServiceRepositoryTest`
Ex03	`@Configuration` + `@Bean` (clasă JDK `Clock`)	`Ex03ClockBeanTest`
Ex04	`@Qualifier` (2 implementări)	`Ex04QualifierTest`
Ex05	`@Value` din `application.yaml`	`Ex05ValueTest`
Ex06	`@Primary`	`Ex06PrimaryTest`
Ex07	`@PostConstruct`	`Ex07PostConstructTest`

Nivel 2 — `fundamentals/practica/p01–p10` (doar cerința)

`// Cerință:` într-o frază, fără pas cu pas. Tu decizi ce adnotări folosești și unde.

#	Subiect	Concepte
P01	`WeatherService` cu unitate din yaml	`@Service` + `@Value` cu default
P02	`Welcomer` cu `Translator` default	interfață + `@Component` + `@Primary`
P03	`DateTimeFormatter` cu pattern din yaml	`@Configuration` + `@Bean` + `@Value`
P04	`EmailSanitizer` cu blacklist la pornire	`@Component` + `@PostConstruct`
P05	`RequestCounter` — instanță nouă la fiecare lookup	`@Scope("prototype")`
P06	`OrderProcessor` combină `Clock` (Ex03) + `WeatherService` (P01)	constructor injection între pachete
P07	`AuditTrail` activ doar pe profilul `audit`	`@Profile` + `@ActiveProfiles`
P08	`InventoryService` injectează un `List<String>` publicat	`@Bean` colecție + `@Qualifier`
P09	`UserRegistrationService` rulează *toți* validatorii	injecție `List<T>`
P10	`Stopwatch` setează `startedAt` la pornire	`@PostConstruct` în state

Nivel 3 — `fundamentals/provocari/c01-c10` (fără hint)

Nicio adnotare, niciun `// TODO`, niciun import pus dinainte. Cerința **funcțională** e în `provocari/CERINTE.pdf`. Testul îți spune ce trebuie să iasă — **tu alegi singur** mecanismul Spring.

Concepte cheie

- **ApplicationContext** — registrul de bean-uri construit de Spring la pornire.
- **Bean** — orice obiect gestionat de Spring (creat, wire-uit, distribuit de container).
- **Component scan** — Spring scanează pachetul clasei `@SpringBootApplication` și sub-pachetele lui. De-aia fiecare zonă (`demo`, `fundamentals`, `notifications`) are propria clasă `...Application`.
- **Constructor injection** — preferat: imutabilitate, dependențe explicite, testabil fără Spring.
- **`@Configuration` + `@Bean`** — sursă explicită de bean-uri pentru clase pe care nu le poți adnota (JDK, third-party).
- **`@Qualifier` / `@Primary`** — alegerea bean-ului când există mai multe de același tip.
- **`@Value`** — citire de valori scalare din `application.yaml`.

Definition of Done

- `./mvnw test` → toate cele 51 de teste GREEN (6 demo + 39 exerciții + 6 challenge).
- Ai parcurs întrebările din `teorie/EXERCITII-TEORIE.pdf`.
- Poți explica:
  - de ce `Clock` are nevoie de `@Bean` și nu de `@Component`;
  - ce eroare aruncă Spring dacă scoți `@Qualifier` când există 2 implementări;
  - diferența dintre `@Qualifier` și `@Primary`;
  - de ce constructor injection e preferat în locul `@Autowired` pe câmp;
  - când rulează `@PostConstruct` (după DI, înainte ca bean-ul să fie folosit).